ARTEFACT · SOFTWARE ENGINEER (SENIOR, BRAZIL)

Written case

Paulo Cezario

Theory and practice of rendering, backend approaches and tRPC, followed by a task manager built with Next.js 15 and a transport-agnostic tRPC backend. The appendix is the project README: how to run, test and verify the result.

Candidate	Paulo Cezario
Role	Software Engineer · Senior · Brazil
Repository	github.com/phscezario/artefact-selective-process
Contents	Theoretical questions · practical case · README
Date	November 10, 2026

Contents

PART 1 — THEORETICAL QUESTIONS

1. React and Next.js: Rendering and Strategies
2. Backend: Concepts and Approaches

PART 2 — PRACTICAL CASE

1. What I built
2. Architecture
3. API and validation
4. Server-side rendering
5. Behaviour and user experience
6. Screenshots
7. Tests and verification
8. Running the application
9. Trade-offs and possible next steps

APPENDIX — PROJECT README

1. Stack
2. How to run
3. Architecture
4. API
5. What is implemented
6. Decisions and trade-offs
7. Tests
8. What I would do next
9. Repository layout

Part 1 — Theoretical Questions

1. React and Next.js: Rendering and Strategies

a) Explain the differences between CSR, SSR and SSG

The main difference between these rendering strategies is where and when the page is generated. This affects the initial user experience, performance, SEO and the resources required to run the application.

Client-Side Rendering (CSR)

With CSR, the browser is responsible for rendering the application. The server initially sends the HTML structure and JavaScript files, and React takes over in the browser, fetching the required data and building the interface.

This approach works particularly well for interactive applications, such as dashboards, administrative systems and platforms where users spend a lot of time interacting with the application.

The main disadvantage is the initial loading experience. Depending on the amount of JavaScript and the data required, users may have to wait before seeing meaningful content. It can also create SEO challenges when the content depends entirely on JavaScript execution.

Server-Side Rendering (SSR)

With SSR, the server generates the HTML for each request, usually using the latest available data, and sends the rendered page to the browser. React then hydrates the page, making its interactive elements functional.

The biggest advantage is that users can see the content earlier, without waiting for the entire client-side application to load. It is also useful for pages that need to be indexed by search engines while displaying frequently changing information.

On the other hand, rendering every request on the server introduces additional processing costs. If the application depends on slow APIs or databases, this can also affect response times.

Static Site Generation (SSG)

With SSG, the HTML is generated during the build process rather than for every user request. The generated files can then be distributed through a CDN.

This is generally a very efficient approach because the server does not need to generate the page every time someone accesses it.

It works particularly well for content that does not change frequently, such as documentation, blogs, landing pages and company websites.

The limitation is that the generated content can become outdated. When information changes, the application needs a new build or a revalidation strategy, such as Incremental Static Regeneration (ISR).

In simple terms, CSR renders in the browser, SSR renders on the server for each request, and SSG generates the page ahead of time.

In Next.js 15, these strategies can be combined depending on the requirements of each route. Server Components, Client Components, caching and dynamic rendering allow developers to choose the most appropriate approach for different parts of the same application.

b) In which scenarios can each approach be advantageous or problematic? Give practical examples.

The choice depends mainly on the type of application, how frequently its data changes, and how users interact with it.

CSR — Client-Side Rendering

A good example would be an internal management platform or an analytics dashboard.

In these applications, users are usually authenticated, interact with several components and frequently update information. SEO is generally not a priority.

CSR can simplify the development of highly interactive interfaces. However, using it for a public website or an e-commerce landing page could negatively affect the initial loading experience and search engine visibility.

SSR — Server-Side Rendering

An e-commerce platform is a good example.

Product information, prices and stock availability can change frequently. Rendering the page on the server allows users to receive updated information while also making the content available in the initial HTML.

The disadvantage appears when traffic increases significantly. Rendering every request can become expensive, particularly when the application depends on external services or database queries.

Caching can help reduce this overhead, but it needs to be designed carefully to avoid serving outdated information.

SSG — Static Site Generation

Documentation websites, blogs and marketing pages are good examples.

Their content usually does not change with every request, so generating the pages during the build process is more efficient.

However, this approach becomes less attractive when the content changes frequently or depends on user-specific information. ISR can help when content needs periodic updates without rebuilding the entire application.

Combining different strategies

In real projects, I would not necessarily choose only one rendering strategy for the entire application.

For example, an e-commerce platform could use SSG for marketing pages, SSR for product pages that require fresh information, and CSR for interactive features such as shopping carts and customer account areas.

I believe the important thing is to understand the requirements of each part of the application before choosing the rendering strategy.

Application to this challenge

For this task management application, SSR is appropriate for the listing page because the requirements explicitly ask for the tasks to be loaded before the page is delivered to the user.

The creation and editing forms, on the other hand, benefit from Client Components because they require user interaction, form state management and event handling.

This separation allows the application to take advantage of server-side rendering without making the entire interface dependent on client-side execution.

---

2. Backend: Concepts and Approaches

a) What are the main ways to implement a backend for a web application?

There are different ways to structure a backend, and I believe the decision should be based on the application's requirements, complexity, integration needs and expected growth.

Some of the most common approaches are:

1. Backend integrated with a full-stack framework

Frameworks such as Next.js, Remix, Django and Laravel allow frontend and backend functionality to live within the same application.

This is a practical approach for applications where a separate backend service is not necessary. It simplifies deployment, reduces infrastructure complexity and can make development faster.

The main limitation is that frontend and backend responsibilities can become tightly coupled if the application grows without clear boundaries.

2. REST APIs

REST is one of the most common approaches for building backend services.

The backend exposes endpoints representing resources, and clients communicate with them using standard HTTP methods such as GET, POST, PUT, PATCH and DELETE.

One of its biggest advantages is flexibility. Different applications, technologies and platforms can consume the same API.

It is also a mature approach with well-established standards for authentication, caching, documentation and monitoring.

The main challenge is maintaining consistency between frontend and backend contracts, especially when the application has many endpoints and different clients.

3. GraphQL

GraphQL allows clients to request exactly the data they need through a query language.

It can be useful when multiple clients consume the same backend but require different data structures, such as web applications, mobile applications and external integrations.

However, it introduces additional complexity around resolvers, authorization, caching, query performance and database access.

4. RPC and typed APIs

RPC approaches expose backend operations as procedures rather than focusing exclusively on resources.

tRPC is a good example for TypeScript applications because it provides end-to-end type safety between the backend and frontend.

This reduces duplicated contracts and makes development more straightforward when both sides are maintained by the same team.

The main limitation is that this approach is more tightly coupled to the technologies used by the client and server.

5. Microservices and event-driven architectures

For larger systems, the backend can be divided into independent services organized around business domains.

Each service can have its own responsibilities, deployment lifecycle and scaling requirements. Communication can happen through HTTP APIs, messaging systems or events.

This approach is useful when different parts of the system have independent scaling, availability or ownership requirements.

However, it also introduces operational complexity, including distributed tracing, service communication, eventual consistency and failure handling.

I have worked with distributed systems and integrations where these concerns are important, particularly when dealing with payment processing and operations that must avoid duplicate execution.

My approach to choosing an architecture

I don't believe there is one approach that works for every application.

For a smaller application, a well-structured backend within a single project is often enough. As business requirements grow, it may become necessary to separate services or introduce different communication strategies.

What I consider important is keeping business rules and responsibilities well organized, so that the application can evolve without requiring unnecessary changes across the entire system.

The architecture should solve the current problem while leaving room for future requirements.

b) Do you know tRPC? What are its main advantages and disadvantages, especially regarding security, typing and scalability?

Yes. tRPC is a framework that allows TypeScript applications to communicate through procedures while maintaining type safety between the backend and frontend.

Instead of manually defining API contracts or generating types from a separate specification, the client can infer types directly from the backend router.

I see this as one of its biggest advantages, especially when frontend and backend development happen within the same project or team.

Advantages

End-to-end type safety

The frontend can infer the expected input and output types directly from the backend procedures.

This reduces duplicated interfaces and helps identify contract changes during development. For example, if a backend procedure changes the structure of its response, TypeScript can identify the affected frontend components.

Simpler validation and maintenance

tRPC works well with validation libraries such as Zod.

We can define the expected input structure once and use it to validate incoming requests on the server. The same schema can also be reused in the frontend to provide immediate feedback before submitting a form.

This helps keep validation rules consistent across the application.

Developer experience

The integration with TanStack Query provides useful features such as caching, loading states, query invalidation and mutations.

It reduces the amount of repetitive code required to manage communication between the frontend and backend.

Less boilerplate

Compared with a traditional REST implementation that requires separately maintained contracts or code generation, tRPC can make development more straightforward when working entirely within the TypeScript ecosystem.

Disadvantages and limitations

Security

Type safety should never be confused with security.

TypeScript types are removed at runtime, which means the backend must still validate every incoming request. A client can send unexpected or malicious data regardless of what the frontend types indicate.

Authentication, authorization, input validation, rate limiting and proper error handling must still be implemented.

For example, a procedure that updates a task should verify that the task exists and, in a real multi-user application, that the current user has permission to modify it.

Technology coupling

One limitation is that tRPC is primarily designed for TypeScript environments.

If the application needs to expose an API to external partners, mobile applications written in different languages or services built with other technologies, REST or GraphQL may be a better option.

Scalability and maintenance

tRPC itself is not necessarily a scalability limitation. It uses HTTP for communication and can run on infrastructure similar to other Node.js applications.

The real considerations are related to the backend architecture, database performance, request volume, caching, resource consumption and deployment strategy.

As the application grows, large routers and tightly coupled frontend/backend contracts can also make maintenance more challenging.

It is important to keep procedures organized by business responsibility and avoid putting all application logic directly inside the routers.

When would I choose tRPC?

I would choose tRPC when the frontend and backend are part of the same TypeScript ecosystem, especially when development speed, maintainability and type safety are important.

For example, it is a good fit for internal platforms, dashboards and applications developed by a single team.

For public APIs, third-party integrations or systems with multiple technologies, I would generally prefer REST or GraphQL because they provide a more independent contract between consumers and the backend.

Ultimately, I see tRPC as a good tool for the right scenario, not as a replacement for every API architecture.

Part 2 — Practical Case

Repository: https://github.com/phscezario/artefact-selective-process

1. What I built

I built a task management application covering all CRUD operations required by the case. The task list is rendered on the server, and I also implemented the infinite-scroll bonus.

The interface and documentation are in English, and the application uses fictional data. It can be started locally with:

```
npm install
npm run dev
```

Technology stack

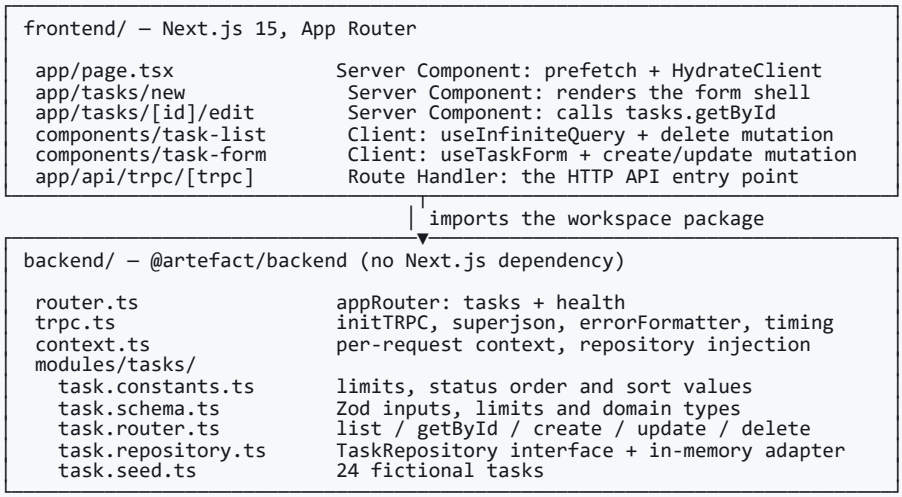
Layer	Technology	Version
Framework	Next.js (App Router, Server Components)	15.5
UI	React / React DOM	19.3
API	tRPC + TanStack React Query integration	11.27 / 5.107
Validation	Zod	4.6
Serialization	superjson	2.2
Tests	Vitest	3.2
Language	TypeScript (`strict` + `noUncheckedIndexedAccess`)	5.9
Monorepo	npm workspaces	npm 10

The case specifies four task fields: `id` , `titulo` (required), `descricao` (optional) and `dataCriacao` . I kept those names as requested, added an optional `status` field (`todo` , `in_progress` or `done`) to make each task's state visible in the list, and a `dataAtualizacao` timestamp that follows the same naming convention.

`dataAtualizacao` starts equal to `dataCriacao` and is bumped on every edit. It gives the list a second sortable dimension (what changed most recently, regardless of when it was created) and lets the row show "Updated" only when that actually happened.

The status defaults to `todo` , so creating a task with only the required fields still works.

2. Architecture



I kept the backend in its own workspace package instead of tying its implementation directly to Next.js. It contains the tRPC router, validation schemas and `TaskRepository` interface, and has no dependency on the frontend framework.

Next.js exposes the router through a single route handler using the tRPC fetch adapter. Server Components can also call the router directly, in-process, so the server does not make an unnecessary HTTP request to its own API.

This keeps the deployment straightforward while maintaining a clear separation between the application and its transport layer. For example, the in-memory repository could be replaced by a PostgreSQL implementation without changing the task procedures. Likewise, the API could later be exposed through another server adapter without moving business rules into the frontend.

3. API and validation

The application exposes the tRPC API through:

```
/api/trpc/*
```

Procedure	Type	Input	Possible errors
`tasks.list`	Query	`{ limit?: 1..50 (default 8), cursor?: string, sort?: TaskSort }`	`BAD_REQUEST`
`tasks.getById`	Query	`{ id }`	`NOT_FOUND`
`tasks.create`	Mutation	`{ titulo, descricao?, status? }`	`BAD_REQUEST`
`tasks.update`	Mutation	`{ id, titulo?, descricao?, status? }` (at least one field)	`BAD_REQUEST`, `NOT_FOUND`
`tasks.delete`	Mutation	`{ id }`	`NOT_FOUND`

Procedure	Type	Input	Possible errors
`health`	Query	No input	—

`TaskSort` is `created-desc` (default), `created-asc`, `updated-desc`, `titulo-asc` or `status-asc`. The value is validated by the same Zod enum the UI uses, so an unsupported order is rejected with `BAD_REQUEST` instead of silently falling back on the server.

The validation rules are defined in `task.schema.ts` and reused by the frontend and backend. The form uses `createTaskInput.safeParse` to provide immediate feedback, while the tRPC procedure validates the request again on the server.

The title is trimmed, required and limited to 120 characters. The description is trimmed and limited to 1,000 characters. An empty description is normalised to `null`, giving the application a consistent representation for a task without a description.

Errors are also handled explicitly. The tRPC `errorFormatter` adds a `fieldErrors` map when validation fails with a `ZodError`. This allows the frontend to identify the field that needs attention instead of displaying only a generic error.

For example, a validation response can include:

```
{
  "error": {
    "json": {
      "data": {
        "fieldErrors": { "titulo": "Title is required." }
      }
    }
  }
}
```

Stack traces are available during development and removed in production. An unexpected server error is replaced by a generic message, so an internal detail never reaches the browser. When a task does not exist, the API returns `NOT_FOUND` with a message explaining that the task may have already been deleted. The edit page uses the same condition to render the Next.js not-found page.

The `dataCriacao` and `dataAtualizacao` fields remain JavaScript `Date` objects on both sides. I use superjson to preserve the date type during transport, avoiding manual conversion from ISO strings in the components.

Pagination

The list endpoint uses cursor-based pagination. The `nextCursor` value contains the ID of the last task in the current page, and the next request continues from that position.

I chose this approach because tasks can be deleted while someone is browsing the list. With offset pagination, deleting an item between requests can shift the remaining records and cause an item to be skipped. Cursor-

based pagination makes this scenario easier to handle. If a cursor refers to a task that has since been deleted, the implementation restarts from the first page.

4. Server-side rendering

The task list is rendered on the server, as required by the case. The request flow is:

1. Next.js renders `app/page.tsx` as a Server Component. The route declares `export const dynamic = 'force-dynamic'` because the in-memory store can change after a mutation. This ensures the list page is rendered dynamically instead of reusing stale static HTML.
2. The page calls `getQueryClient().prefetchInfiniteQuery(trpc.tasks.list.infiniteQueryOptions(...))`. `getQueryClient` uses React `cache`, allowing the page and nested components to share the QueryClient for that request.
3. The router is called in-process, so there is no HTTP round trip from the server to its own tRPC endpoint.
4. `HydrateClient` dehydrates the prefetched query data and passes it to the client tree through a `HydrationBoundary`.
5. The initial HTML already contains the task rows, status badges and formatted creation dates. The browser does not need to fetch the first page before displaying the list.
6. On the client, `TaskList` uses the same `infiniteQueryOptions`. TanStack Query can use the hydrated data for the initial render. The query is configured with a 30-second `staleTime` and `refetchOnWindowFocus: false`.
7. Create, update and delete operations run through `/api/trpc`. After a successful mutation, the relevant task-list query is invalidated so the displayed data and total are refreshed.

I also kept date formatting consistent between the server and browser. Dates use the `en-US` locale and the UTC time zone, avoiding differences caused by the server and user's local time zones during hydration.

The create page is a Server Component that renders the client-side form. The edit page retrieves the task on the server through `caller.tasks.getById` and passes its values to the form as `initialValues`. If the task no longer exists, the page calls `notFound()` rather than rendering an invalid form.

5. Behaviour and user experience

The list displays the task title, description, status and creation date, with eight items loaded per page. When a task has been edited, the row also shows when it was last updated.

Ordering

The list can be ordered by creation (newest or oldest), last update, title or status. The chosen order lives in the URL (`/?sort=updated-desc`), so the server component prefetches exactly that view and the link can be shared or bookmarked.

The seed data makes this visible: eight of the 24 tasks were edited after being created, so "recently updated" produces a different order than "newest created". Status uses the workflow order (`todo` , `in_progress` , `done`) instead of alphabetical order, and title sorting ignores case.

Each order is implemented as a total order with the task `id` as the tiebreaker. Without it, two tasks with the same timestamp could swap places between two page requests.

Infinite scroll

An `IntersectionObserver` watches a sentinel near the bottom of the list. Its `rootMargin` is set to `240px`, so the next page can start loading before the user reaches the end.

The sentinel is only rendered while another page is available. Once all tasks have been loaded, the interface displays a message indicating that the end of the list has been reached.

The observer is updated when the fetching state changes to avoid triggering duplicate requests.

Loading, empty and error states

- **Loading:** `app/loading.tsx` displays skeleton rows during navigation. Infinite scrolling has a separate inline loading indicator.
- **Empty state:** When there are no tasks, the page displays a message and an action to create the first task.
- **Error state:** If loading fails, the list displays an error message and a retry action.

Delete flow

Deleting a task requires an inline confirmation instead of using `window.confirm`. The selected row displays its own pending state while the request is running.

On success, the application displays a confirmation toast and invalidates the list query. If the API returns an error, an error toast is displayed instead. Keeping confirmation within the row avoids blocking the entire interface.

Create and edit forms

Both operations use the same `TaskForm` component, with separate create and edit modes. Form state is managed with React hooks through `useTaskForm`, without adding a form library for this relatively small case.

The form includes:

- Client-side validation using the same Zod schema as the API.
- Field-level error messages, with `aria-invalid` and `aria-describedby`.
- Server validation errors mapped back to the relevant fields.
- A submitting state on the button, including a `Saving…` label and `aria-busy`.
- Success feedback, query invalidation and navigation back to the list.
- A character counter for the description and `maxLength` on both fields.
- `useId()` to associate input labels and IDs correctly.

I also included accessibility details such as semantic landmarks, visible focus indicators, `role="alert"` for field errors, `aria-live` for toast messages and support for `prefers-reduced-motion`.

The layout is responsive, with row actions moving below the task content on smaller screens.

6. Screenshots

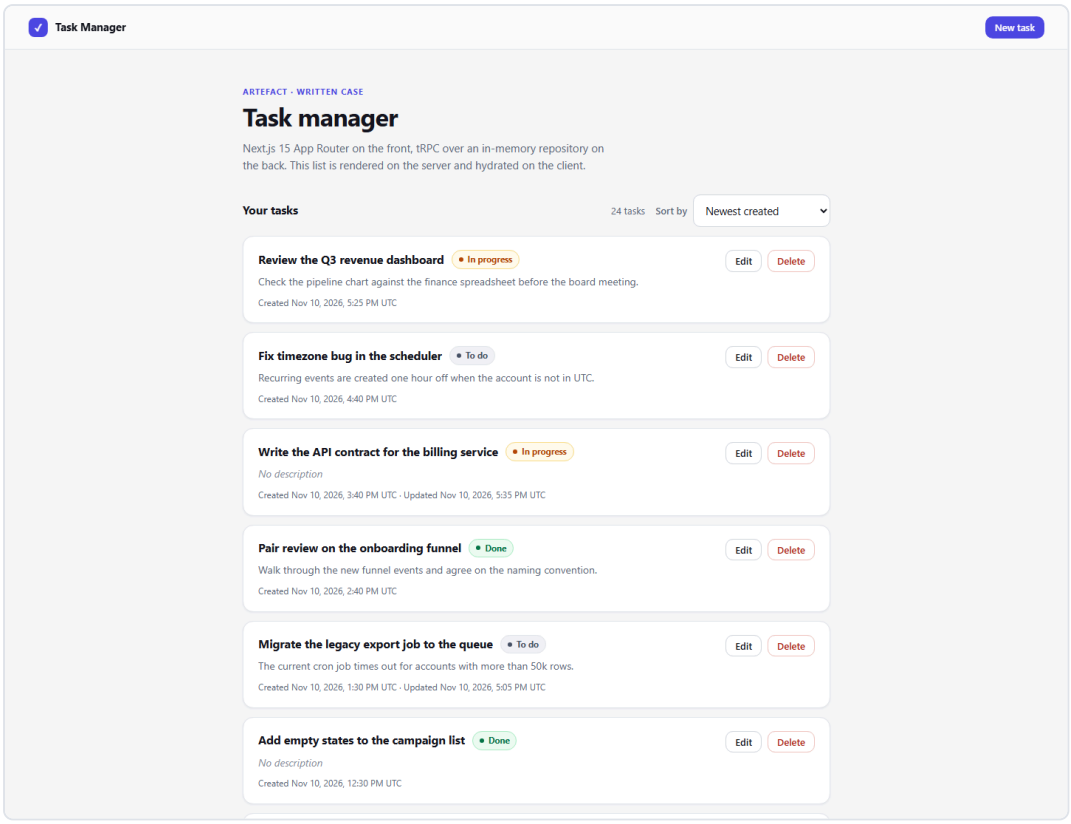


Figure 1 — The list page is server-rendered, and the initial response already contains the tasks.

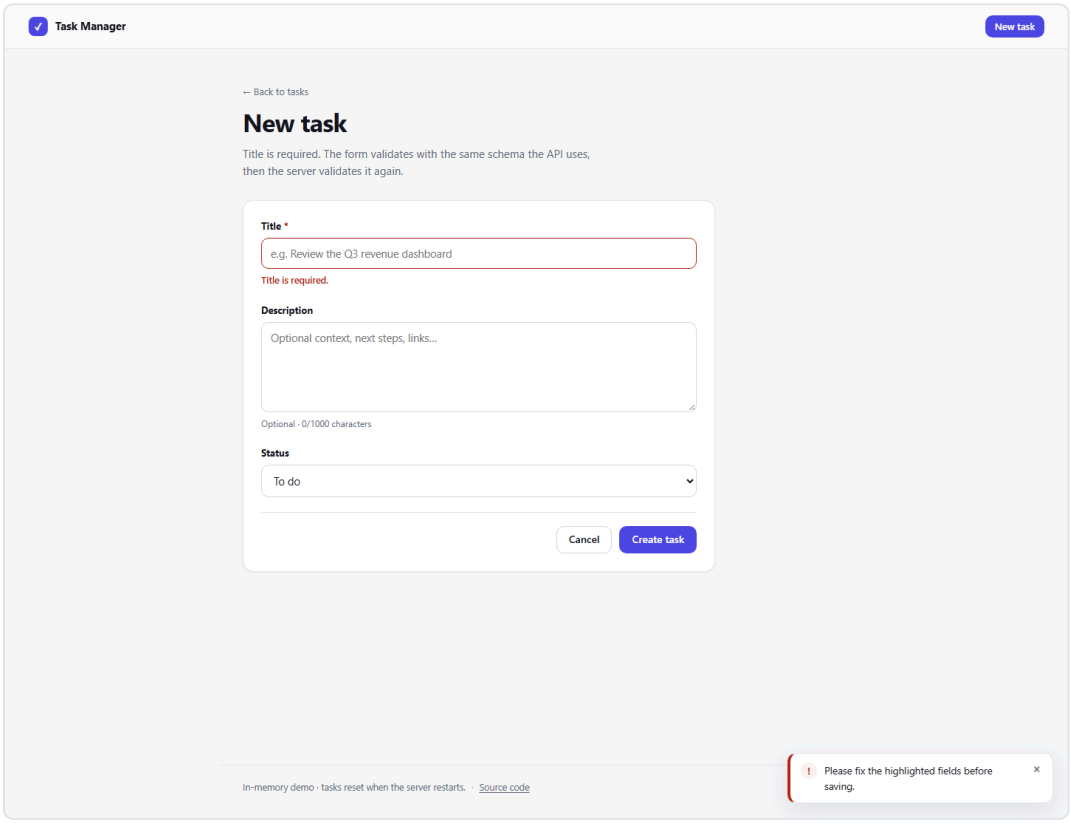


Figure 2 — Client-side validation displays the field error and an error toast.

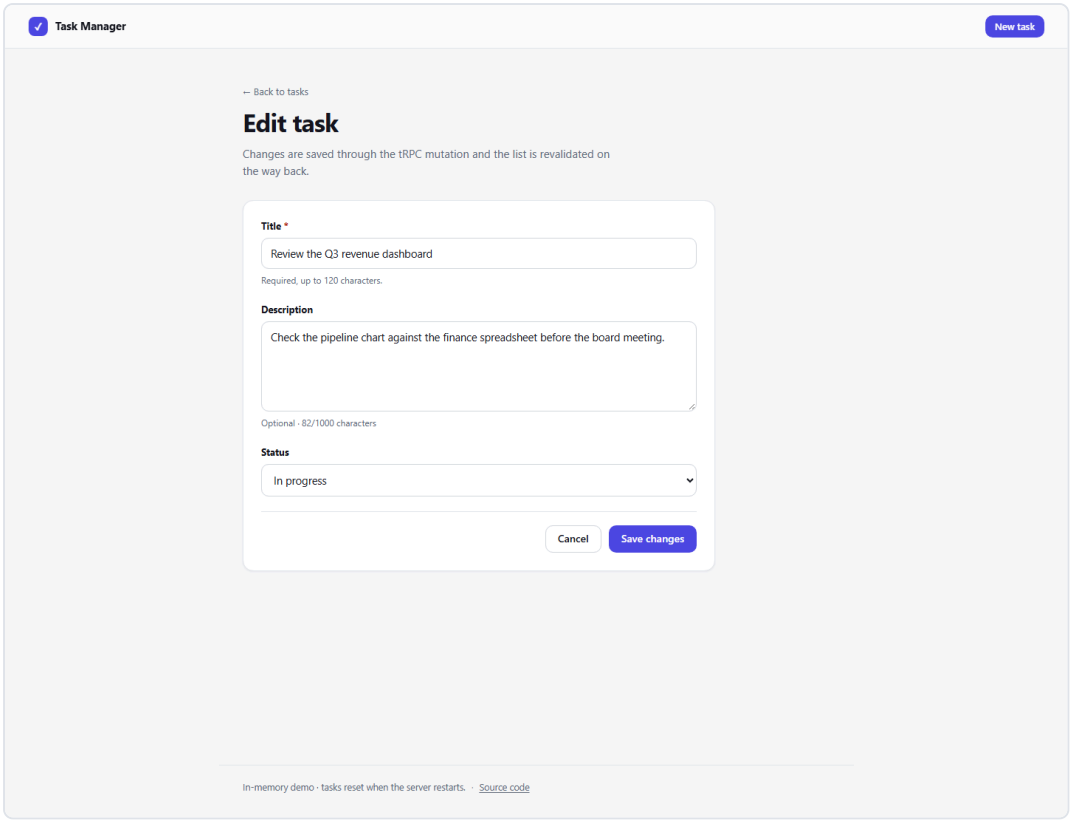


Figure 3 — The edit form receives its initial values from the server, while the mutation runs on the client.

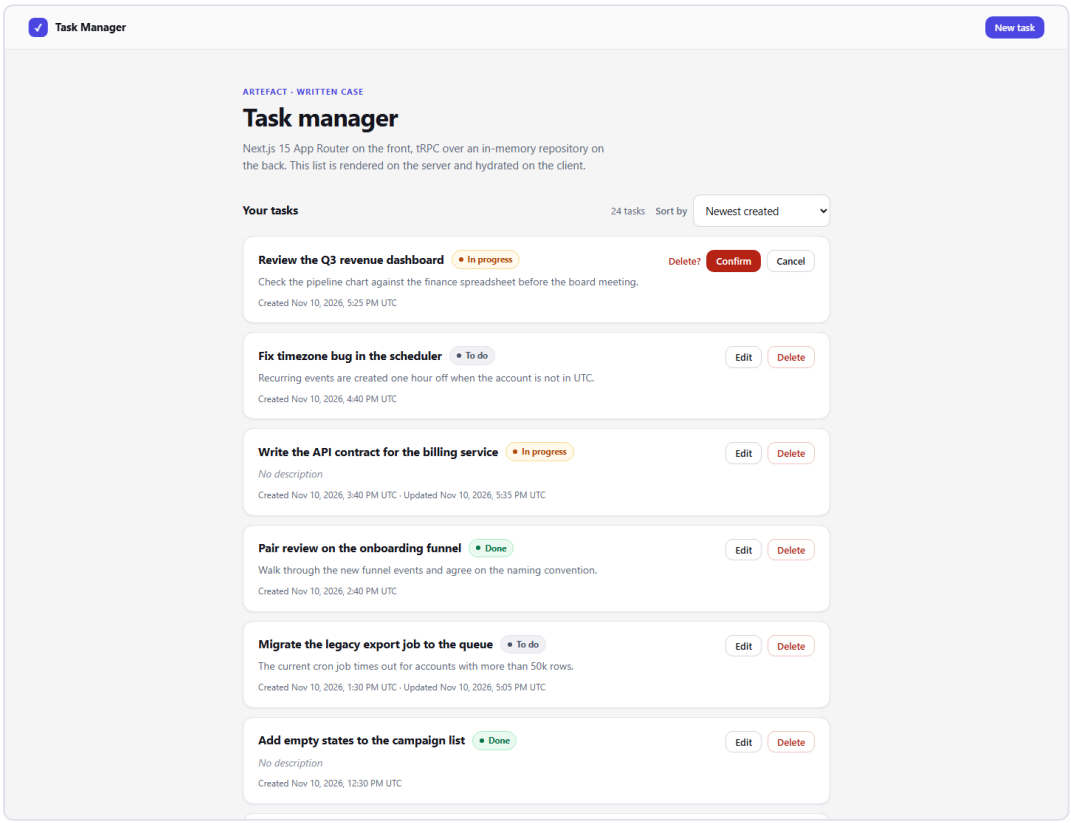


Figure 4 — The user confirms deletion inside the row, without a browser dialog.

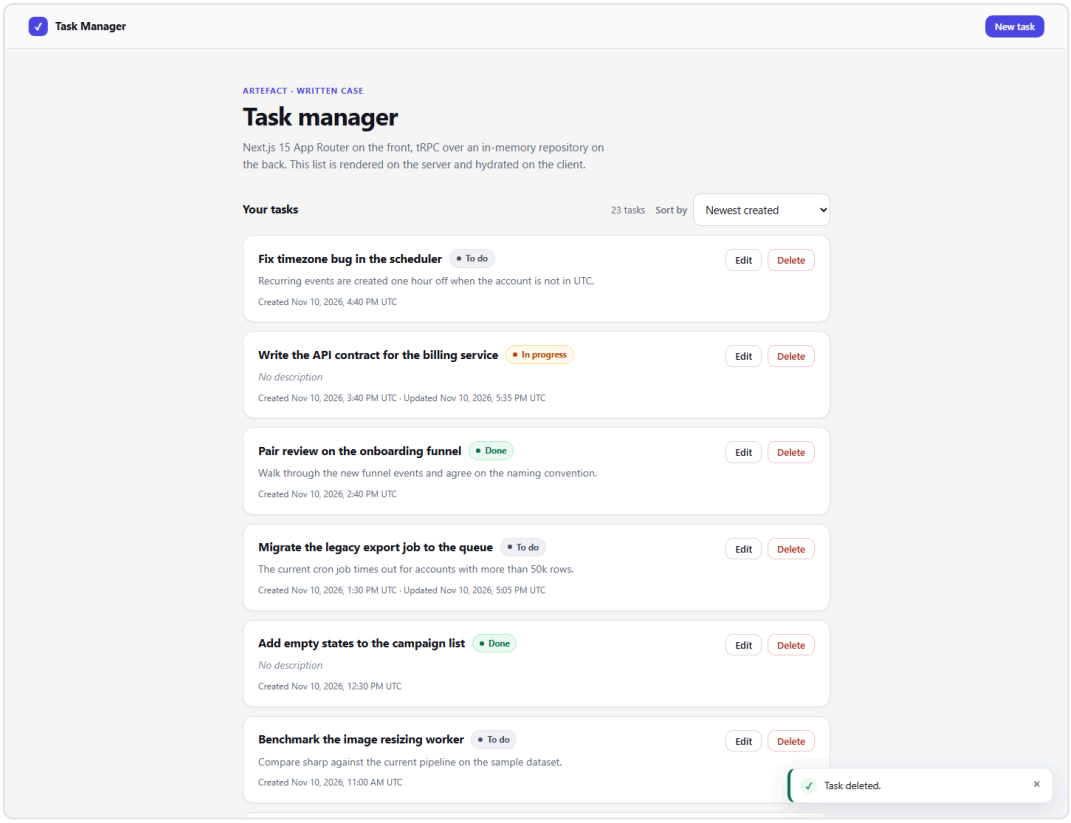


Figure 5 — Success feedback is displayed and the list is refreshed.

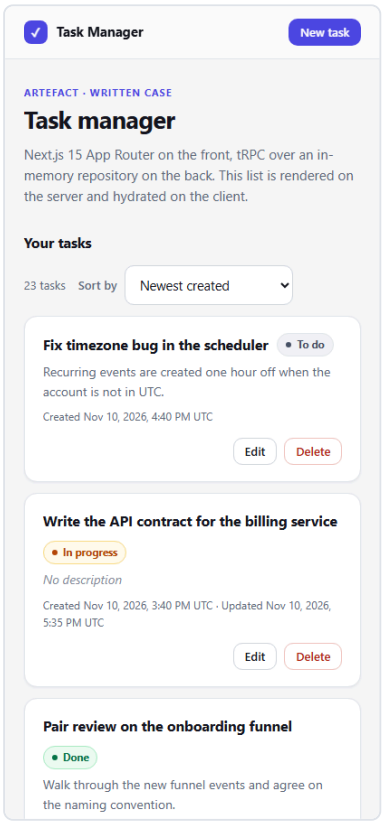


Figure 6 — On smaller screens, the task actions move below the task content.

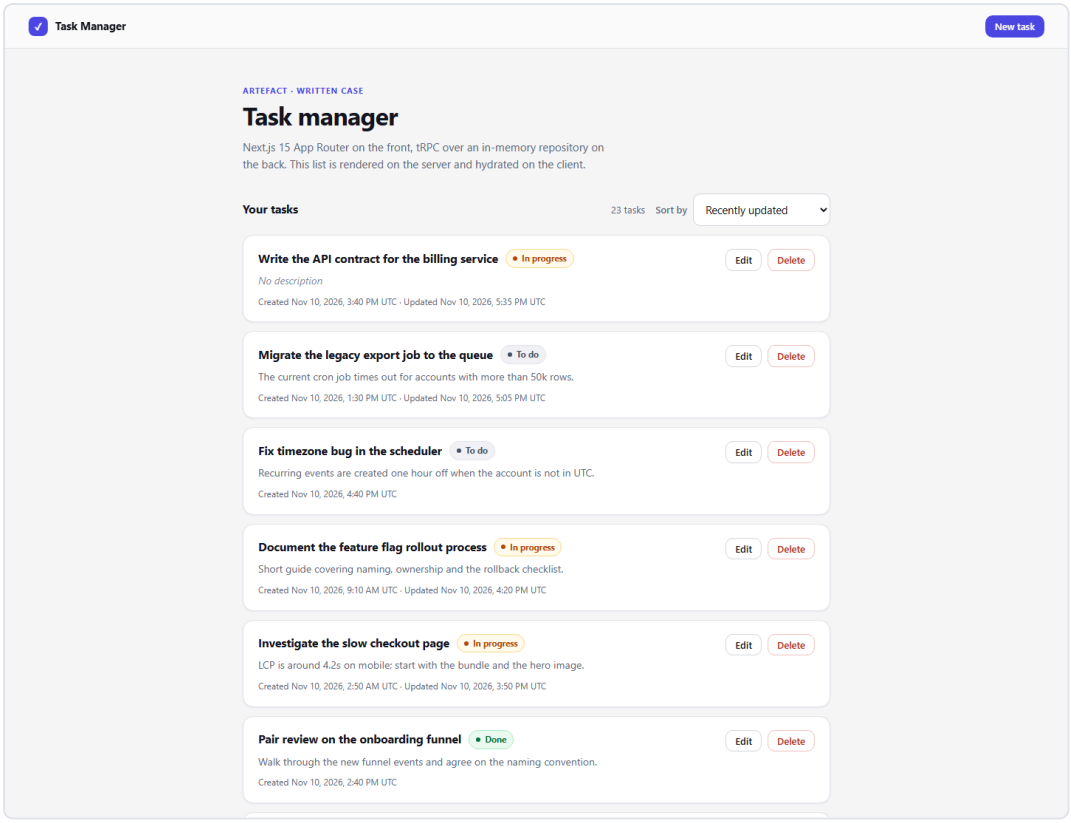


Figure 7 — Ordering by last update, chosen in the list and reflected in the URL.

7. Tests and verification

The project provides a verification command:

```
npm run verify
```

It runs ESLint, a Prettier check, the TypeScript checks, the Vitest suites of both workspaces and the Next.js production build. CI runs this same command, so the local gate and the pipeline cannot drift.

The backend has 33 tests. Most of them exercise the real router with an injected repository, without HTTP or mocks. Four go through the same `fetchRequestHandler` adapter the app mounts at `/api/trpc`, because that is the only place where the promises of the wire can actually be verified: `dataCriacao` and `dataAtualizacao` arrive as `Date` objects (superjson) and a rejected input carries `fieldErrors`.

The tests cover:

1. Creating a task generates a unique ID, trims the title and normalises a whitespace-only description to `null`; the status defaults to `todo` when omitted, and both timestamps start equal.
2. An empty title is rejected with `BAD_REQUEST`; the exact title limit is accepted and one character more is rejected.
3. The same boundary check for the description (1000 accepted, 1001 rejected).
4. An unknown status is rejected.
5. Cursor pagination across three pages returns every item exactly once and ends with `nextCursor: null`; the list is ordered newest-first.

6. An unknown cursor falls back to the first page, and the limit defaults to 8 when omitted.
7. An invalid page size (0 or 51) is rejected.
8. Updating a task changes only the fields included in the request, trims the title, and rejects an update without any field.
9. Updating keeps `dataCriacao` and bumps `dataAtualizacao`.
10. An empty description clears the existing description.
11. Reading, updating and deleting a task that does not exist return `NOT_FOUND`.
12. Deleting a task succeeds once and returns `NOT_FOUND` when attempted again.
13. The seed data has 24 tasks, keeps both timestamps as `Date` objects and passes the entity schema.
14. A task returned to a caller cannot be used to mutate the store.
15. Sorting works for creation (both directions), last update, title (case-insensitive) and status (workflow order), and the default order is unchanged.
16. An unsupported sort value is rejected, and pagination keeps the requested order without repeating or skipping an item.
17. With the seeded data, ordering by creation and by last update produce different sequences.
18. Over HTTP: the dates survive the wire, invalid input returns `BAD_REQUEST` with `fieldErrors.titulo`, a missing task returns 404, and in production an unexpected error exposes neither its message nor its stack.

The frontend has 13 tests for the pure logic behind the list, the form and the error handling: the sort parser (including unknown values in the URL), the shared list input, the field-error mapping and the message extraction from a `TRPCClientError`.

I also checked the production build manually:

- **HTTP smoke test:** Against `npm start`, I tested create, read, update, list, delete, repeated delete and empty-title validation. I checked title trimming, description normalisation, date deserialization, `dataAtualizacao` being bumped by an update, `NOT_FOUND` on the second delete and `BAD_REQUEST` with `fieldErrors.titulo` for an empty title.
- **SSR check:** I inspected the raw HTML returned for `/` and for `/?sort=updated-desc`, and confirmed that each one starts with the task expected for that order. This verifies that the ordering is applied on the server and that the task content is present in the initial HTML response rather than being added only after a client-side request.
- **Infinite-scroll check:** Using Chrome DevTools Protocol, I confirmed that 16 rows were present after hydration and all 24 were loaded after scrolling, followed by the end-of-list message.

The `npm run verify` command completes successfully. The CI workflow runs the same command.

8. Running the application

For local development:

```
npm install
npm run dev
```

The application is available at `http://localhost:3000`.

To run the production build:

```
npm run build
npm start
```

To run with Docker:

```
docker compose up --build
```

The Docker setup uses a multi-stage build based on `node:22-alpine`. It separates dependency installation, the build stage and the runtime image. The final image contains the Next.js standalone output and starts it with `node frontend/server.js`, without including the source files, development dependencies or npm cache.

The CI workflow in `.github/workflows/ci.yml` runs on pushes and pull requests to `main`:

```
npm ci → npm run verify
```

The full verification command is also available locally:

```
npm run verify
```

9. Trade-offs and possible next steps

Some decisions were made to keep the implementation aligned with the scope of the case. They also have limitations that would matter in a production application.

Decision	Reason	Trade-off
Backend as a workspace package	Keeps one deployment while separating backend responsibilities from Next.js	The backend is not independently deployable yet
tRPC instead of REST	Provides end-to-end types without maintaining a separate API contract	Less suitable for non-TypeScript clients and public integrations
Shared Zod schemas	Keeps frontend and backend validation consistent	The validation schema is also included in the client bundle
superjson	Preserves `Date` values across the API boundary	Adds some serialization overhead
Cursor-based pagination	Handles changes to the list more predictably while users browse	The cursor is an implementation detail that the UI does not interpret
Sorting evaluated per request	One implementation covers every order without extra indexes or caching	Editing an item between two pages can move it out of the current cursor window

Decision	Reason	Trade-off
In-memory repository behind an interface	Meets the requirements without introducing a database	Data is lost on restart, and the store is limited to a single application instance
Hand-written CSS	Avoids adding another dependency for a small application	Reusable styling utilities would be useful as the interface grows

If I had more time to extend the project, I would prioritise the following:

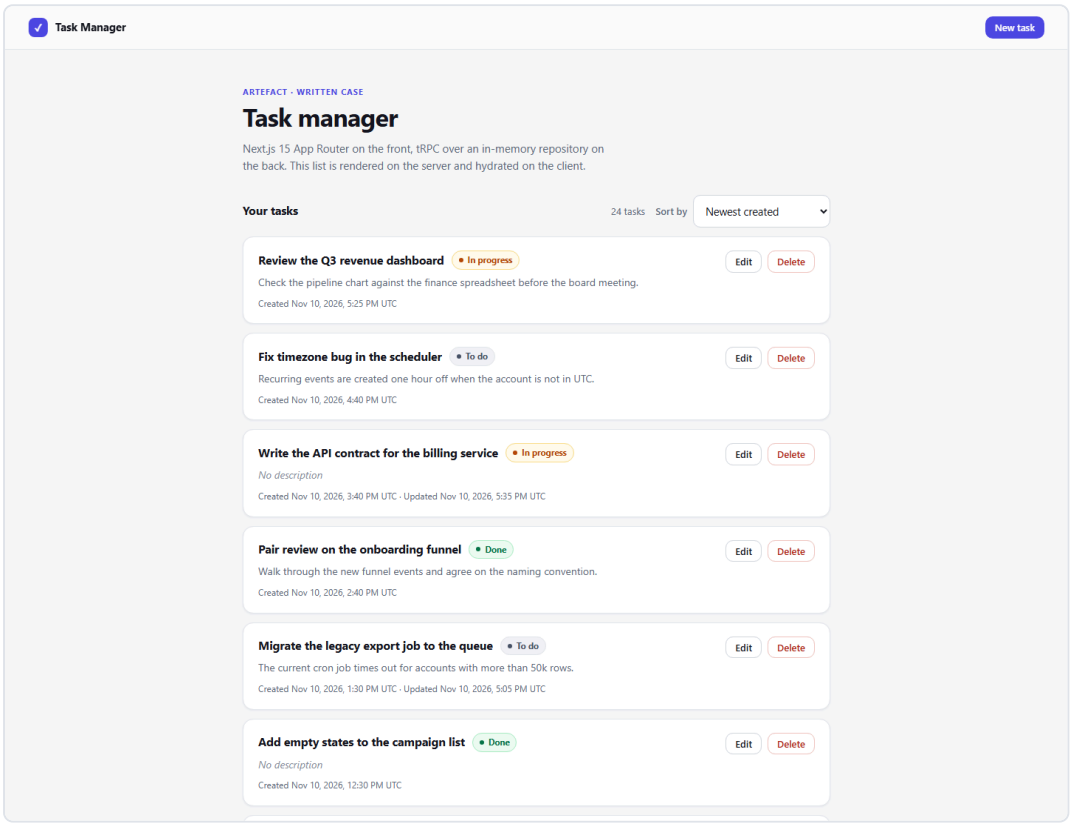
1. Add PostgreSQL through another `TaskRepository` implementation. The existing interface and context injection provide a starting point for this change.
2. Add authentication through `createContext`, which already receives the request headers.
3. Introduce rate limiting at the procedure level.
4. Consider optimistic updates for task deletion.
5. Add Playwright tests for the main user flows.
6. Replace the basic timing log with OpenTelemetry instrumentation.
7. Add a task detail page with support for comments.

For the scope of this case, I focused on implementing the requested functionality, keeping the code organised and making the main flows verifiable without introducing infrastructure that the application does not need.

Appendix — Project README

A small task manager built for the **Software Engineer (Senior, Brazil)** written case: **Next.js 15 App Router** on the front, **tRPC** over an **in-memory repository** on the back, with server-side rendering on the list page and an infinite-scroll bonus.

> The full written answers (theory + practical) and the project README are in `docs/answers`, rendered as a single submission PDF (`Paulo Cezario - Software Engineer Sênior Brazil.pdf`). `npm run submission` rebuilds it from this README and those answers.



Stack

Layer	Choice	Version
Framework	Next.js (App Router)	15.5.29
UI	React / React DOM	19.3
API	tRPC + TanStack React Query integration	11.27 / 5.107
Validation	Zod (shared by server and form)	4.6
Serialization	superjson (keeps `Date` intact on the wire and across RSC)	2.2
Tests	Vitest	3.2

Layer	Choice	Version
Language	TypeScript (strict)	5.9
Monorepo	npm workspaces	npm 10+

How to run

```
# Node 22+ (see .nvmrc)
npm install
npm run dev          # http://localhost:3000
```

Production mode:

```
npm run build
npm start
```

Docker:

```
docker compose up --build   # http://localhost:3000
```

Quality gate (the same command CI runs):

```
npm run verify       # lint -> format:check -> typecheck -> test -> build
```

Submission document (needs Chrome or Edge installed):

```
npm run submission   # docs/answers + README -> one PDF (submission/ and docs/answers/)
```

Architecture

```
┌──────────────────────────────────────────────────────────────┐
│ frontend/  (Next.js 15, App Router)                          │
│                                                              │
│ app/page.tsx ── server component ── prefetch + HydrateClient │
│ app/tasks/new ─┐                                             │
│ app/tasks/[id]/edit ─┴─ server component ── caller.tasks.getById │
│ components/* ── client components ── useInfiniteQuery / useMutation │
│ app/api/trpc/[trpc]/route.ts ── fetch adapter (only HTTP surface) │
└──────────────────────────────┬───────────────────────────────┘
                               │ imports (workspace package, type-safe)
                               ▼
┌──────────────────────────────────────────────────────────────┐
│ backend/  (@artefact/backend — no Next.js dependency)        │
│                                                              │
│ router.ts ── tasks router + health                           │
│ trpc.ts   ── initTRPC, superjson, error formatter, timing    │
│ context.ts ── per-request context (repository injection)     │
│ modules/tasks/                                               │
│   task.constants.ts  limits and sort values (Zod-free)       │
│   task.schema.ts     Zod inputs, entity schema, field errors │
│   task.router.ts     list / getById / create / update / delete │
│   task.repository.ts TaskRepository port + in-memory adapter │
└──────────────────────────────────────────────────────────────┘
```

The backend is a **package, not a process**: the router has no knowledge of Next.js and receives its repository through the context. Mounting it on Fastify/Express later is a ~20-line adapter, and the server components call it **in-process** — no HTTP round trip to your own API on SSR.

API

Single endpoint: `POST|GET /api/trpc/*` (fetch adapter). Procedures:

Procedure	Type	Input	Errors
`tasks.list`	query	`{ limit?: 1..50 (default 8), cursor?: string, sort? }`	`BAD_REQUEST`
`tasks.getById`	query	`{ id }`	`NOT_FOUND`
`tasks.create`	mutation	`{ titulo, descricao?, status? }`	`BAD_REQUEST`
`tasks.update`	mutation	`{ id, titulo?, descricao?, status? }` (at least one field)	`BAD_REQUEST`, `NOT_FOUND`
`tasks.delete`	mutation	`{ id }`	`NOT_FOUND`
`health`	query	—	—

Field names (`titulo`, `descricao`, `dataCriacao`, `dataAtualizacao`) follow the case specification; everything else in the codebase is in English. `dataCriacao` and `dataAtualizacao` are real `Date` objects on both sides (superjson). A task starts with both timestamps equal, and every update bumps `dataAtualizacao` without touching `dataCriacao`.

`sort` accepts `created-desc` (default), `created-asc`, `updated-desc`, `titulo-asc` and `status-asc`; an unsupported value is a `BAD_REQUEST`. Every order is a total order with the task id as tiebreaker, so two pages never disagree about the position of equal timestamps.

Pagination is **keyset**: `nextCursor` is the id of the last item, and the next page starts after it. With `offset/take`, deleting a row between two requests would skip an item; with a cursor the only consequence is that a deleted cursor restarts from the first page.

What is implemented

- **SSR list** — `app/page.tsx` prefetches the first page in a server component, dehydrates the cache and hydrates the client component. `dynamic = 'force-dynamic'` because the in-memory store changes per request (otherwise Next would serve build-time HTML).
- **Ordering** — creation (both directions), last update, title and status. The order lives in the URL (`/?sort=updated-desc`), so the server prefetches exactly that view, the page is shareable and the change is a `startTransition` navigation instead of a client-side reshuffle.
- **Infinite scroll** — TanStack `useInfiniteQuery` + `IntersectionObserver` sentinel with `rootMargin: 240px`; the sentinel only exists while `hasNextPage` is true.
- **Delete with feedback** — inline two-step confirmation (no `window.confirm`), per-row pending state, success/error toasts, and query invalidation of every cached page.

- **Create / edit** — plain hooks (`useTaskForm`), validation with the same Zod schema the API uses, server-side field errors merged back into the form, loading state on submit, redirect + toast.
- **Errors** — `NOT_FOUND` for missing tasks, `BAD_REQUEST` with `fieldErrors` for invalid input, stacks stripped in production, `TRPCClientError` normalised into one readable message.
- **States** — loading skeleton, empty state, error state with retry, end-of-list marker.
- **Seed** — 24 fictional tasks so pagination and the infinite scroll are visible immediately.

Decisions and trade-offs

Decision	Why	Cost
Backend as a workspace package instead of a second server	One command to run, no CORS, SSR calls the router in-process, but the domain still has a hard boundary	Backend isn't independently deployable *today* (the router already is transport-agnostic)
tRPC instead of REST	End-to-end types with zero codegen; the same Zod schema validates the form and the API	Not consumable by non-TypeScript clients; one URL makes per-resource CDN caching unnatural
Zod as the single source of validation	No rule drift between client and server	The schema ships to the client (~13 KB gzipped)
superjson	`dataCriacao` stays a `Date` instead of a stringly-typed field	Small payload overhead
Keyset pagination	Correct under concurrent deletes	Cursor is opaque to the UI
Sort per request	One comparator covers every order, no extra indexes	Editing an item between two pages can move it out of the current cursor window
In-memory repository behind an interface	The case does not need a database; the domain does not care either	Data resets on restart, single instance only
Hand-written CSS	No build step or extra dependency for a case this size	No utility classes for future screens
`npm` workspaces with TypeScript sources	No build/watch step for the backend package	Consumers need `transpilePackages` (configured)

Tests

```
npm test        # vitest run in both workspaces
```

33 backend tests. Twenty-nine of them drive the real router with an injected repository (no HTTP, no mocks of the code under test); four go through the same `fetchRequestHandler` adapter the app mounts, which is the only way to verify what the wire actually promises: `dataCriacao` and `dataAtualizacao` as `Date` objects (superjson), `fieldErrors` on `BAD_REQUEST`, and a production `INTERNAL_SERVER_ERROR` that leaks neither message nor stack. They cover creation with trimmed title and normalised description, both length boundaries, an unknown status, keyset pagination with ordering and no duplicates, an unknown cursor, the default page size, partial updates, the update timestamp, every sort order, `NOT_FOUND` on update/delete/getById, defensive copies, and the 24 seeded tasks.

13 frontend tests cover the pure logic behind the list, the form and the error handling: the sort parser (including unknown values in the URL), the shared list input, the field-error mapping and message extraction

from a `TRPCClientError`.

The production build was also smoke-tested end to end (create → update → list → delete → double delete → empty title), including the SSR HTML assertion that seeded tasks are present in the first response.

What I would do next

Persistence (Postgres + a second `TaskRepository`), authentication in `createContext` (the HTTP context already carries the request headers; the RSC caller would resolve them with `next/headers`), per-procedure rate limiting, optimistic updates on delete, Playwright for the critical flow, OpenTelemetry spans instead of the timing `console.info`, and a `/tasks/[id]` detail route.

Repository layout

```
backend/    domain + tRPC router (no Next.js dependency)
frontend/   Next.js app: route handler, server components, client components
docs/
  answers/          written answers (Part 1 and Part 2) + the submission PDF
  screenshots/      images used by the README and by the PDF
scripts/            submission PDF builder (Markdown -> HTML -> Chrome headless)
submission/         generated PDF + HTML (gitignored)
```